At the simplest level, you can secure parts of your application, so only authenticated users can access them. You will start off by learning how to do that for single pages, or endpoints. You will also look at securing all pages in a specific folder or across the entire application. Then you will explore some of the features provided by ASP.NET Core Identity for managing authorization on a more granular level. You will look at grouping users together according to their *role* and managing access to endpoints on that basis. Then you will explore the notion of claims, introduced in the last chapter, in more detail and establish how you can formulate authorization policies based on claims and apply them for managing access to endpoints. Preventing access to endpoints is known as *request-based* authorization. While you learn how to create policies, you will understand how they are based on requirements and their handlers. You will write your own requirements and handlers and use them to formulate some policies.

Finally, you will look at implementing fine-grained authorization within a page. For example, many users may be authorized to access a page that lists properties to rent, but only selected users will be authorized to edit them. In that scenario, you do not want to present the Edit navigation to those users who do not have sufficient permissions. Applying authorization policies within a page like this is known as *resource-based* authorization.

I’ve always thought that authorization shares something in common with dependency injection. Both are actually relatively simple topics to understand, but both are shrouded in a fog of abstract concepts. Just as I hope to have lifted the fog in the chapter on dependency injection, I’m going to start by explaining some of the concepts surrounding authorization you will be working with.

10.1 Basic authorization in Razor Pages